

Practical – 5

Practical Title: Registration Form with Frontend Validation and User-Friendly Error Handling

Problem Definition: Create a student registration form with HTML5 input types and JavaScript validation for name, email, mobile number, password, confirm password, course, year, gender, and terms acceptance. Use Regular Expression for validations.

A. Theory

Q1. Are correct input types and attributes used?

Appropriate HTML5 input types should be used according to the type of information being collected. For example, email fields can use `type="email"`, password fields can use `type="password"`, and mobile numbers can use an appropriate telephone input.

Attributes such as required, labels, and meaningful field names help users understand the information expected by the registration form.

Q2. Are validation errors displayed near the relevant fields?

Yes. JavaScript validation should identify the field containing invalid data and display a clear error message associated with that field.

This makes it easier for the user to understand what needs to be corrected instead of displaying a general error message for the entire form.

Q3. Is password strength checked?

Password validation checks whether the entered password follows the defined requirements. Regular Expressions can be used to verify the required combination of characters.

The validation prevents weak or incorrectly formatted passwords from being accepted by the frontend.

Q4. Is the form accessible using keyboard and screen-reader-friendly labels?

The form should use standard HTML form controls, properly associated labels, and a logical field order so that users can navigate it using a keyboard.

Clear labels and meaningful error messages also improve accessibility for users who depend on assistive technologies.

B. Implementation/ Output/Result:

The **StudentHub Portal registration form** was enhanced with client-side validation using **JavaScript and Regular Expressions**. The validation was integrated into the existing `script.js` file

The registration form is accessed through `register.html` and contains fields such as **Name, Student ID, Email, Phone Number, Program, Semester, and Password**. JavaScript selects these elements from the DOM using `getElementById()` and reads their values when the user enters or submits data.

Regular Expressions are used to validate important fields. Separate validation patterns are applied to the **name, Student ID, email, mobile number, and password** fields so that incorrectly formatted data can be identified. Required fields are also checked to ensure that the user does not submit an incomplete registration form.

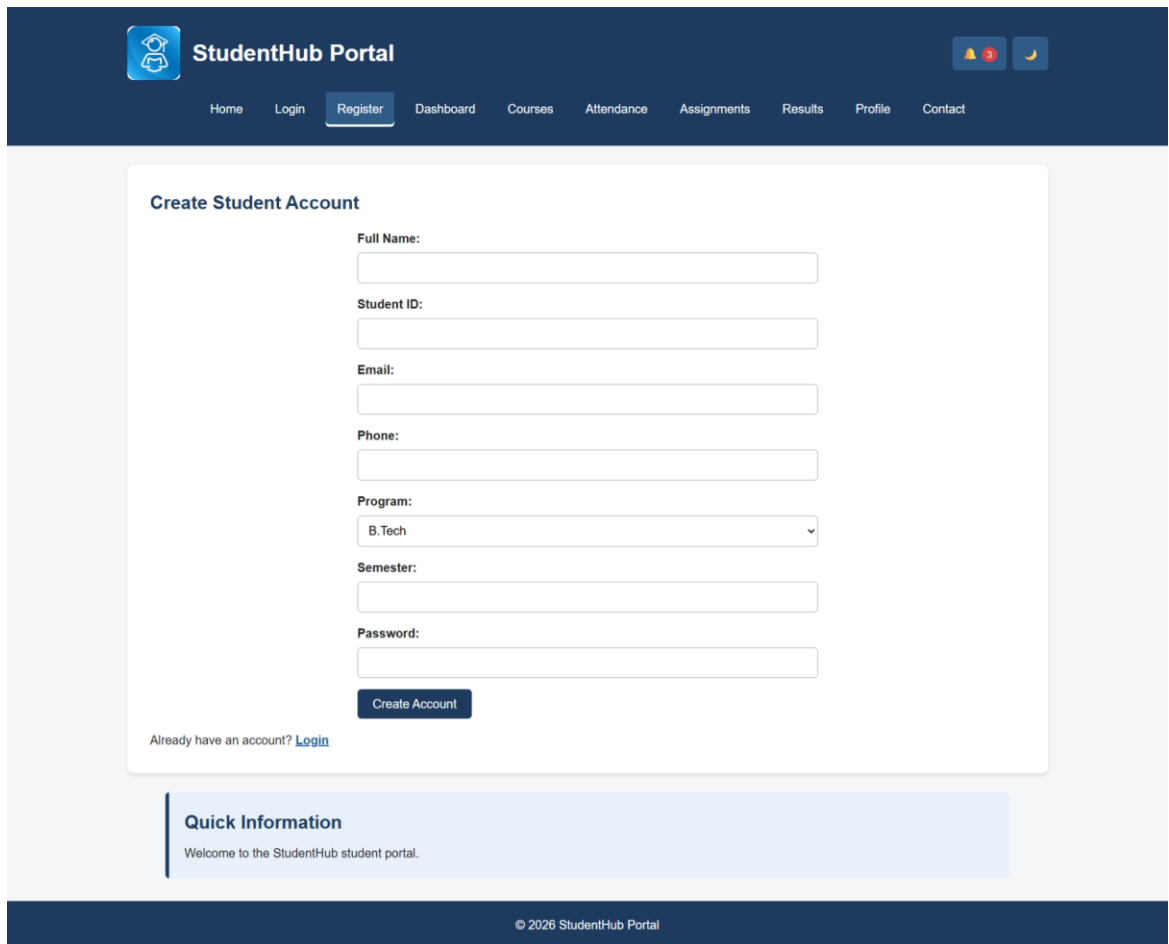
Event handling is used to provide validation feedback during form interaction. Validation functions are triggered when the user leaves or changes relevant fields and again when the registration form is submitted. If an input is invalid, a clear error message is displayed so that the user knows which information needs to be corrected. Valid inputs are allowed to proceed without unnecessary error messages.

A reusable error-handling approach is used to keep the JavaScript code organized and reduce repeated validation logic. The form submission is prevented when invalid data is detected, ensuring that incorrect information is not submitted through the frontend.

The Practical 5 validation functionality was added without removing the existing Practical 4 features. The same `script.js` continues to handle the **hamburger menu, light/dark theme switching with localStorage, active navigation, FAQ accordion, modal popup, slider, notification banner, notification counter, assignment search/filter, and registration validation**. This demonstrates how new functionality can be integrated into an existing JavaScript-based web project.

The implementation was tested using both **valid and invalid registration data**. Invalid inputs such as empty fields, incorrectly formatted email addresses, invalid phone numbers, incorrect Student IDs, and weak/incorrect passwords were tested to verify that appropriate validation messages were generated. Valid inputs were also tested to confirm that the form behaves correctly when the entered information satisfies the defined rules.

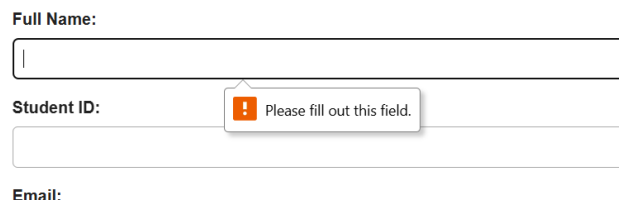
C. Evidence of Work (Screenshots):



The screenshot shows the 'StudentHub Portal' registration page. The header is dark blue with the portal logo and name on the left, and notification and theme icons on the right. A navigation menu below the header includes links for Home, Login, Register (highlighted), Dashboard, Courses, Attendance, Assignments, Results, Profile, and Contact. The main content area is titled 'Create Student Account' and contains a form with the following fields: Full Name, Student ID, Email, Phone, Program (a dropdown menu currently showing 'B.Tech'), Semester, and Password. A 'Create Account' button is positioned below the password field. At the bottom of the form, there is a link: 'Already have an account? [Login](#)'. Below the form is a 'Quick Information' section with the text 'Welcome to the StudentHub student portal.' The footer is dark blue and contains the copyright notice '© 2026 StudentHub Portal'.

Fig1: Register

Create Student Account



This close-up view of the registration form shows the 'Full Name' and 'Student ID' fields. The 'Full Name' field is empty. The 'Student ID' field is also empty, and a validation message box is displayed over it, containing an orange exclamation mark icon and the text 'Please fill out this field.' The 'Email' field is partially visible below.

Fig2: Validation Message

Email:

aryan.gmail.com

Phone



Please include an '@' in the email address. 'aryan.gmail.com' is missing an '@'.

44566

Fig3: Email

This page says

Password must contain at least 6 characters.

OK

Fig4: Password

This page says

Account details validated successfully!

OK

Fig5: Valid Message

D. GitHub Repository Link:

[aryan150806-wq/StudentHubPortal: Web Development Framework P](#)

E. Challenges Faced:

- **Writing validation rules:** Creating suitable Regular Expressions for fields such as name, email, mobile number, Student ID, and password required careful understanding of input formats.
- **Handling different types of invalid input:** Testing empty fields, incorrect email formats, invalid mobile numbers, and weak passwords helped identify and fix validation errors.

- **Connecting JavaScript with HTML:** Selecting the correct form elements using DOM methods and attaching event listeners correctly was initially challenging.
- **Displaying user-friendly errors:** Instead of allowing incorrect data to be submitted, validation messages had to be shown clearly near the relevant fields without affecting the existing page design.
- **Maintaining existing functionality:** Since the registration validation was added to the existing StudentHub script.js, care was required to avoid disturbing the Practical 4 features such as theme switching, navigation, FAQ, modal, and slider.

F. Learning Outcomes:

T I learned how to create and validate a student registration form using **HTML5 and JavaScript**. I understood the use of **DOM elements, event handling, and Regular Expressions** to check whether the entered data is valid or invalid.

I also learned how to provide **clear user-friendly validation messages** and test forms using different valid and invalid inputs. This practical improved my understanding of **client-side validation, accessibility, and better user experience** in web applications.

Faculty Remarks: